

Dual-Arm Robotic Assembly of Interlocking Brick Structures

Master Project Report & Executable Build Specification

Author: Sakib · HKUST, Dept. of Mechanical & Aerospace Engineering **Version:** 3.0 — 15 September 2026
Supersedes: plan v1 (MuJoCo), plan v2 (Isaac Lab + Newton), literature validation report **Status:** Normative.
This document is the single source of truth for the project.

PART 0 — HOW TO USE THIS DOCUMENT

0.1 Purpose

This document serves three audiences simultaneously:

- 1. **A human researcher** who needs to understand what is being built and why each decision was made.
- 2. **AI coding agents** executing work packages, which need unambiguous inputs, outputs, and acceptance tests.
- 3. **An orchestrator** (human or agent) that schedules work, checks gates, and decides when to cut scope.

It is written so that the project can be **recreated from scratch** by someone with no prior conversation context.

0.2 Normative language

Term	Meaning
MUST	Non-negotiable. Violating this invalidates the results or breaks the schedule.
SHOULD	Strong default. Deviate only with a recorded reason in the project ledger.
MAY	Genuine option. Pick based on measurement.
DO NOT	A known failure mode. Previously attempted or documented as wasteful.

0.3 The single most important instruction

Four modules of this project already exist as published, open-source work. The dominant failure mode for this project is rebuilding them. Before writing code for any work package, an agent **MUST** check §1.2 ("Already solved — do not rebuild") and confirm the component is not on that list.

The project's contribution is **not** a LEGO assembly system. It is the **quantified relationship between robotic structural bracing and contact-rich insertion performance** in a long-horizon discrete assembly task. Everything else is infrastructure to be borrowed.

0.4 Document map

Part	Contents	Primary audience
I	Literature foundation, what's solved, what's contested, what's open	Human, orchestrator
II	System specification: architecture, physics, platform, schemas	Agents
III	Work packages WP0–WP8 with acceptance tests	Agents
IV	Evaluation and experiment protocol	Human, analyst agent
V	AI orchestration: task graph, agent roles, prompt templates, ledger	Orchestrator
VI	Risk register and contingency tree	Orchestrator
VII	Appendices: reward spec, hyperparameters, glossary, full bibliography	All

0.5 Known unresolved question

This project has two possible framings, and they have different deliverables:

- **(A) Research project** — simulation-only, contribution is the bracing/insertion result. This document assumes (A).
- **(B) ISDN5240 RoboFab course project** — the course template asks for "the object or installation you plan to fabricate," implying a physical artifact.

If (B) applies, §3 (Work Packages) still holds but a physical execution phase must be appended, and the schedule in §5.2 does not fit. **Resolve this with the course instructor before WP0 begins.** Do not assume simulation-only is acceptable for a fabrication course.

PART I — LITERATURE FOUNDATION

1.1 Review method

The review covered four bodies of work: (a) contact-rich robotic assembly and RL insertion; (b) computational brick/LEGO design, stability and sequencing; (c) bimanual and cooperative robotic assembly; (d) simulation of press-fit and snap-fit mechanics. Sources were retrieved through academic search across Semantic Scholar, PubMed, Scopus and arXiv indexes, supplemented by targeted web verification of platform and project claims. Every claim below traces to a citation in §7.4.

The review was conducted **after** an initial plan was drafted, specifically to test that plan. It falsified four of its components. Those falsifications are recorded here rather than quietly removed, because they are the most useful part of the document.

1.2 Already solved — DO NOT REBUILD

1.2.1 Snap-fit physics simulation → BrickSim

[BrickSim] is a real-time physics simulator for interlocking brick assemblies, built on Isaac Sim and open-sourced. Its stated motivation is that existing rigid-body simulators do not faithfully capture snap-fit mechanics. It introduces a compact force-based mechanics model for snap-fit connections, solves internal

force distribution with a structured convex quadratic program, and uses a hybrid architecture that delegates rigid-body dynamics to the underlying physics engine while handling snap-fit mechanics separately. Reported: 100% accuracy in static stability prediction across 150 real assemblies at ~5 ms average solve time, and faithful reproduction of both the occurrence and location of real structural collapse in dynamic drop tests.

Implication. The plan's original "write a Warp clutch kernel in Week 1" is superseded. Two things matter:

1. The **hybrid architecture** (engine for rigid bodies, separate module for the joint) was independently arrived at and is now externally validated. Keep the architecture.
2. The **per-connection spring model** originally specified is inferior. BrickSim solves the internal force distribution across the *whole assembly*. This matters directly: when the placer arm presses a brick into a structure, the reaction propagates through every clutched joint below. A per-connection model cannot say which joint fails first. A global QP can — and that prediction is exactly what the bracing arm needs.

Action: WP1 is *evaluate and integrate BrickSim*, with a fallback to extending it (see §1.4.2), not reimplementing it.

Repository: github.com/intelligent-control-lab/BrickSim

1.2.2 Structural stability analysis → StableLego

[StableLego] formulates stability as an optimization over force-balancing equations for 3D block-stacking structures. It correctly predicts whether a structure is stable and — critically for this project — **accurately locates the weakest parts of a design**, outperforming prior heuristic methods. It ships a dataset of 50,000+ 3D objects with LEGO layouts and stability labels.

Implication. The originally planned graph-articulation/min-cut heuristic is the weaker approach. [Legolization] moved the field from heuristics to force-based analysis in 2015; StableLego is the current form. Additionally, target structures for evaluation **SHOULD** be sampled from StableLego rather than hand-authored — this removes authoring effort and gives the evaluation set external provenance that reviewers will prefer.

1.2.3 Assembly sequence planning → physics-aware action masking

[PhysASP] learns a construction policy with an online physics-aware action mask that filters invalid actions, applied to LEGO assembly across more than 250 3D structures with a 100% success rate, where the best comparable baseline failed on more than 40 structures.

Implication. The originally planned greedy topological sort with lookahead is the baseline this beats. Use the action-masking formulation, or cite it and justify a simpler method on runtime grounds — but do not present a greedy sequencer as novel.

Related and worth reading: **[ASPGraph]** (graph-transformer ASP with a LEGO dataset), **[BudgetBrick]** (sparse 3D CNN with a one-initialized convolution filter for constraint validation — an elegant trick), **[KollskerLEGO3D]** / **[KollskerLEGO2D]** (MIP + adaptive large neighbourhood search with a QP static-equilibrium check, scaling to ~77,000 brick positions).

1.2.4 Generative brick design from text or image → BrickGPT and successors

[BrickGPT] generates physically stable brick structures from text via an autoregressive LLM with an efficient validity check and physics-aware rollback during inference that prunes infeasible token predictions using physics laws and assembly constraints. It releases StableText2Lego (47,000+ structures) and demonstrates the designs assembled by robotic arms.

The originally proposed stretch goal — "LLM emits JSON, validator checks overlap/connectivity/stability, feed violations back, cap at 5 iterations" — **is BrickGPT's published method**. Worse, **[RollbackFree]** has already superseded it by moving physical-validity enforcement from test-time rejection sampling to training-time RL with assembly-level rewards, achieving rollback-free generation orders of magnitude faster.

[BrickAnything] and **[LegoACE]** cover geometry- and multi-view-conditioned generation; **[BrickNet]** covers graph-based build sequences over thousands of real part types.

Action: DELETE both the image → structure and text → structure stretch goals. Use BrickGPT or StableLego as an *input source* for target structures. The generation problem is crowded and moving fast; the manipulation problem is where the contribution lies.

1.2.5 Gripper-collision-aware LEGO sequencing → already done

[BricksToBots] integrates gripper/structure collision avoidance into LEGO sequence planning with a two-finger gripper, using an assembly-by-disassembly algorithm, instance segmentation combined with stud detection, and named placement techniques ("Safe Approach," "Wiggling Technique") on a 3-DoF delta robot.

Correction on record: the earlier plan asserted that the coupling between gripper geometry and build plan was "genuinely under-explored" and should be claimed as a contribution. **That claim is false and MUST NOT appear in any writeup.** Cite BricksToBots. The differentiation must come from the dual-arm and force-control dimensions, not from this coupling.

1.3 Validated — keep as specified

Design decision	Supporting evidence
Two-phase coarse transport → precision insertion	[IndustReal], [Maroungkas]
Policy-level action integrator (accumulate pose targets, not absolute commands)	[IndustReal]
Sampling-based curriculum over initial pose error	[IndustReal], [SnapCurriculum]
Task-space impedance / variable compliance action space	[VarCompliance] (193 cites), [MultiModalImpedance], [FuzzyVTS]
Domain randomization over friction, mass, gains, latency	[FORGE], [MultiModalImpedance]
Asymmetric actor-critic with privileged critic	[ProvablePriv], [InformedAAC], [ACGD]
Stabilizer + placer dual-arm role split	[BUDS] , [ManiGaussian++], [A3D]
Explicit joint model rather than emergent contact	[BrickSim] , [LammleSnapFit], [YoonTightToI]

Two of these deserve elaboration because they carry the project's framing.

[BUDS] "**Stabilize to Act**" states the role-assignment framing almost verbatim: a stabilizing arm holds an object in place to simplify the environment while an acting arm executes the task, motivated by the high dimensionality of the bimanual action space. BUDS achieves 76.9% success across four tasks from 20 demonstrations and is **56.0% more successful than an unstructured baseline** — attributed specifically to the precision these tasks require. That last clause is the strongest available support for this architecture: structured role assignment wins *because the task demands precision*. LEGO insertion is a precision task.

MUST: cite BUDS as the paradigm being instantiated. **DO NOT** present stabilizer+placer as this project's idea.

[FORGE] is the closest published system to this project. It combines a force-threshold mechanism with dynamics randomization for robust transfer under significant pose uncertainty; policies are **conditioned on a maximum allowable force** and adaptively perform contact-rich tasks while avoiding aggressive behaviour regardless of controller gains; they predict task success for efficient termination and autonomous threshold tuning. It demonstrates **forceful insertion of snap-fit connectors**.

Two mechanisms to adopt from FORGE that the original plan lacked:

1. **Condition the policy on a force budget** rather than only penalizing force in the reward. This addresses the "slam-and-snap" failure mode by construction instead of by reward tuning.
2. **Learn a success predictor** as an auxiliary head, replacing the hand-coded snap-verification threshold and enabling early termination.

1.4 Contested — these require ablation, not assumption

1.4.1 Is vision necessary at close range?

The literature genuinely disagrees, and the disagreement is load-bearing for the schedule.

Evidence that force alone suffices:

- **[MarkertDQN]**: a state vector consisting **only of F/T signals from the tooltip, with no position information**, reaching **100% success at 0.2 mm clearance** via offline-trained DQN.
- **[SymmetrySoftWrist]** (ICRA 2024): a partially observable formulation with a memory-based agent acting **purely on haptic and proprioceptive signals**, comparable to or outperforming a state-based agent.
- **[PolyFit]**: abandons RL entirely for **F/T-based supervised learning** of extrinsic pose, reporting 96.7% and 91.3% real-world success on unseen polygon shapes.
- **[CaolInsertion]**: F/T-based insertion succeeding when **pose uncertainty exceeds task clearance by more than 10×**.

Evidence that multimodal helps: [MultiModalImpedance], [VisionTouch].

Physical argument specific to this task: in the final 3 mm of a LEGO insertion, the brick and gripper occlude the socket. A wrist camera is looking at the back of the brick it is holding.

MUST: run ablation A3 (no vision) in WP5, **before** building the perception module in WP6. WP6 is conditional on A3's outcome.

1.4.2 Dense reward shaping vs. sparse reward with a model-based prior

[Marougkas] (ICRA 2025) addresses sub-millimetre insertion, noting that recent RL efforts *often depend on careful definition of dense reward functions*. Instead they use a potential-field model-based controller under full observability, integrated with **residual RL trained on only a sparse, goal-reaching reward**, with a curriculum over observation noise and action magnitude. Trained purely in simulation, zero-shot transferred, and it **outperforms recent RL-based methods in this domain and prior hybrid-policy efforts**.

Implication. The originally specified six-term hand-weighted reward may be over-engineered. **MUST:** make residual RL the *primary* run and end-to-end dense-reward RL the ablation, not the reverse. Add a sparse-reward variant of the residual run. This also de-risks the schedule, since residual converges faster.

1.4.3 Is the fixture-arm approach correct at all?

[Stavridis] (IROS 2018) argues explicitly **against** using one arm as a fixture: rather than utilizing one arm as a fixture for holding one of the parts while the other performs the assembly, they propose motion generation in the relative end-effector frame involving both arms, with a task-priority strategy optimizing motion and force capabilities.

MUST: address this in the writeup rather than ignore it. The counter-argument for this task: in LEGO assembly the base is rigidly fixed to a baseplate, so relative-motion generation has nothing to gain — the sub-assembly cannot be repositioned into a more manipulable configuration. State this explicitly; it demonstrates the opposing view was read.

1.4.4 Multi-fingered hand for the placer

[DualArmDexPiH] (RA-L 2022) implements peg-in-hole using dual arms *and* dexterous robotic hands, with "advanced blind grasping," in-hand manipulation for workpiece reorientation, feed-forward task-space force control, and a four-stage assembly strategy built from "perturbation pattern" unit motions, on a 50-DoF upper-body robot.

Correction on record: the earlier claim that a multi-fingered hand is "the wrong tool" for the placer is too strong — it demonstrably works. The defensible claim is narrower: a hand **adds a sample-complexity and contact-count burden disproportionate to this project's compute and time budget**. That is a scoping argument, not a technical one, and must be phrased as such.

Keep the hand on the **stabilizer**, where [A3D]'s affordance framing supports it, and cite DualArmDexPiH when noting placer-side hands are viable but out of scope.

Related: **[CompliantFingers]** (RA-L 2025) optimizes compliant fingertip geometry in simulation against task-level success, increasing tolerable workpiece variation by 2.29×. Their sim-to-real finding is a caution: **failure during insertion has a small sim-to-real gap, whereas failure during search and in-hand slip has a much larger one**. Do not over-trust simulated grasp-stability results.

1.5 Documented hazards

1.5.1 Teacher–student distillation can fail structurally

The paradigm is well-supported, and **[ProvablePriv]** gives theoretical grounding including polynomial sample and computational complexity for expert distillation under a "deterministic filter condition." But the same paper identifies **a pitfall of expert distillation in finding near-optimal policies**, and two recent papers name the practical version:

- **[StudentInformedTeacher]**: the student may be **unable to imitate the teacher because of partial observability**, since the teacher is trained without considering whether the student can imitate the learned behaviour. Fix: joint training, penalizing the teacher for the approximated teacher–student action difference plus a supervised alignment step.
- **[RealizableStudents]** (IROS 2025): existing approaches either produce realizable-but-suboptimal teachers or force the student to explore missing information alone; both inefficient. Fix: the student queries the teacher adaptively and resets from recovery states.

Live risk here. A privileged critic that sees true brick-to-socket relative pose and per-stud engagement depth is teaching a behaviour a vision+F/T student cannot reproduce during the final millimetres, when the brick occludes the socket.

MUST: use the **informed asymmetric actor-critic** framing from **[InformedAAC]** — privileged signals need not be full state; any state-dependent privileged signal yields unbiased policy gradients, and carefully selected signals can match or outperform full-state baselines while using strictly less information. Choose critic privileges the student can plausibly *infer* from force history.

1.5.2 Vision distillation is compute-hungry

[VIRAL] is the most directly comparable vision-distillation recipe: privileged RL teacher on full state with a delta action space, vision student distilled via large-scale simulation with tiled rendering, trained with a mixture of online DAgger and behaviour cloning. Their central finding: **compute scale is critical — scaling simulation to tens of GPUs (up to 64) makes both teacher and student training reliable, while low-compute regimes often fail.**

This project has one RTX 5090D. The original estimate of 512–1024 vision environments and ~6 hours for the vision run is **probably optimistic by a wide margin**. This is the strongest argument for running the no-vision ablation first and possibly skipping the vision phase.

1.5.3 The analytic snap-fit theory already exists

The originally proposed "calibrate the analytic clutch against a VBD deformable thin-shell reference" was framed as the novel contribution. **[MechanicsSnapFit]** (*Physical Review Letters*, 2020) analyzes precisely the relevant configuration — **a rigid cylinder and a thin elastic shell** — combining theory, simulation and experiment, constructing a phase diagram over geometric parameters, identifying four distinct mechanical phases, and deriving analytical predictions from linear elasticity theory combined with static friction. It explains the **operational asymmetry** of snap fits: easy to assemble, difficult to disassemble, emerging from the interaction of geometry, elasticity and friction.

A LEGO stud is a rigid cylinder; a LEGO cavity wall is a thin elastic shell. This paper supplies a closed-form model, the insertion/extraction asymmetry the clutch model must reproduce, and a phase diagram to validate against — **without running a deformable simulation at all.**

Further, **[LammleSnapFit]** (2022) already established the "rigid sim + physical joining model" pattern, extending rigid multi-body simulation with physical joining models specifically because established engines cannot simulate deformation during snap-fit assembly, and validating that the extension generates valid RL training data from standard rigid-body simulations. **[YoonTightTol]** takes the data-driven route with real experimental validation.

Action: demote the deformable-reference work. Calibrate against the analytic model first; use deformable simulation only to check the analytic model's regime of validity, with a 3-day hard stop.

For empirical magnitude checks, the cantilever snap-fit FEA literature reports insertion forces spanning roughly 3–35 N and retention forces 1.7–41 N depending on beam geometry, with simulation-to-experiment errors under ~2.4% for well-designed models ([SnapFitFEA1], [SnapFitFEA2]). The 8–15 N per stud figure used in this project sits inside that envelope and **MUST** be cited rather than asserted.

1.6 Open — the actual contribution

Four gaps survived the review, ranked by defensibility.

(1) Quantified bracing for interlocking-brick assembly. [BUDS] established the role split qualitatively. [A3D] learns *where* to support for furniture parts using dense point-level geometric representations with an adaptive module that adjusts support strategy from interaction feedback as the assembly state evolves. [StableLego] locates weak joints. [BrickSim] computes internal force distribution. **Nobody has closed the loop: brace at the joint the force model predicts will fail, and measure the effect on insertion success and peak force across a multi-brick assembly.**

The benchmark to beat is [DualArmSnapFit], which reports **over 96% engagement-detection recall and up to 30% reduction in peak impact forces** versus standard impedance control, using SnapNet to detect snap-fit engagement from joint-velocity transients with proprioception only, plus a dynamical-systems dual-arm coordination framework with event-triggered impedance modulation. Their differences from this project: they use a coupled dynamical system rather than a learned policy; they do one snap-fit at a time rather than long-horizon assembly where every insertion changes the structure being braced; and their arms coordinate with selective decoupling during insertion rather than one arm actively resisting a reaction at a computed weak joint.

This is the primary contribution. Ablation A6 is promoted from ablation to main experiment.

(2) GPU-batched snap mechanics at RL scale. If BrickSim's convex QP at ~5 ms per solve does not batch to thousands of parallel RL environments, a Warp-kernel formulation that does is a real systems contribution — with fidelity comparison against BrickSim as the validation. **Determined in WP1. This decides whether the project has one contribution or two.**

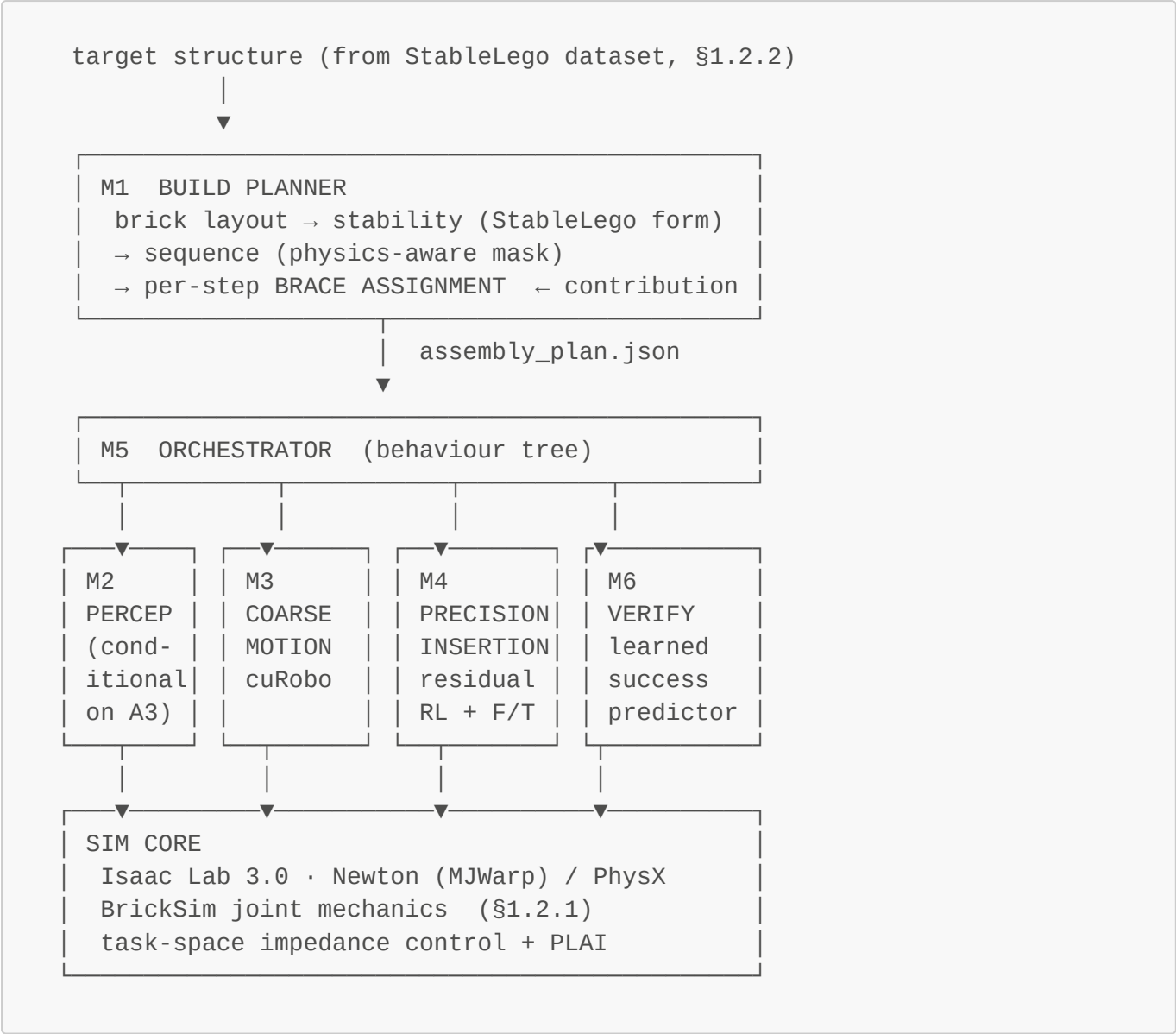
(3) Does snap-model fidelity change what the policy learns? (Ablation A10.) BrickSim validates stability *prediction* against reality. Nobody has asked whether a higher-fidelity mating model produces measurably different learned insertion behaviour — different force profiles, different search strategies. A positive result has implications well beyond LEGO; a negative result is a useful statement about how much simulation fidelity contact-rich RL actually requires.

(4) Force-only vs. multimodal at sub-millimetre clearance under self-occlusion. §1.4.1 shows the literature disagrees and this task has a clean physical argument for occlusion. Publishable on its own.

Note: none of (1)–(4) requires the build planner, the perception module, or generative input. That is where to cut when the schedule slips.

PART II — SYSTEM SPECIFICATION

2.1 System architecture



Interface discipline. Every module communicates only through the typed JSON contracts in §2.6. A module MUST be stubbable with ground-truth data and swappable without touching its consumers. This is what makes the project parallelizable across a human and multiple coding agents.

2.2 Locked decisions with rationale traceability

ID	Decision	Rationale	Source
D1	Isaac Lab 3.0 Beta 2 + Isaac Sim 6.0.1, pinned	Multi-backend, cuRobo integrated, Factory/FORGE envs available	verified 2026-09-15
D2	Newton (MJWarp solver) primary, PhysX fallback by config flag	Warp-kernel access to batched engine state; differentiable; VBD available	§1.2.1, §2.4
D3	BrickSim for joint mechanics; extend only if it fails to batch	Already validated on 150 real assemblies	[BrickSim]

ID	Decision	Rationale	Source
D4	Direct RL env (not manager-based)	Lower per-step overhead; CUDA-graph support; matches Factory template	D1
D5	cuRobo MotionGen for all coarse motion	Already installed; GPU batch IK replaces RRT and accelerates accessibility checks	§2.5
D6	<code>rsL_rl</code> PPO with student-teacher distillation	Native to Isaac Lab; used by the Newton Franka/KUKA-Allegro stacking tasks	D1
D7	Placer arm = Franka + thin parallel-jaw fingertips	Press-fit needs axial force, not dexterity; scoping argument	§1.4.4
D8	Stabilizer arm = Franka/KUKA + Allegro hand	Conforming brace resists multi-directional reaction	[A3D], §1.4.4
D9	Residual RL is the primary run; end-to-end dense reward is the ablation	Sparse reward + model-based prior outperformed dense RL	[Maroungkas]
D10	Force-budget conditioning, not only force penalty	Removes slam-and-snap by construction	[FORGE]
D11	Learned success predictor replaces threshold verification	Enables early termination and autonomous tuning	[FORGE]
D12	Informed asymmetric AC; critic privileges must be student-inferable	Teacher unrealizability is a documented failure	[InformedAAC], [StudentInformedTeacher]
D13	Ablation A3 (no vision) runs before the perception module is built	Literature contested; single-GPU compute limit	§1.4.1, [VIRAL]
D14	Target structures sampled from StableLego, not hand-authored	External provenance; removes authoring effort	[StableLego]
D15	Generative input (image → or text → structure) deleted	Published and already superseded	§1.2.4
D16	Analytic calibration against [MechanicsSnapFit]; deformable sim optional, 3-day cap	Closed-form theory already exists	§1.5.3

2.3 Domain physics — LEGO System geometry

Platform-independent. These numbers are normative.

Quantity	Value	Note
Stud pitch P	8.00 mm	
Brick height	9.60 mm	= 3 plates
Plate height	3.20 mm	
Stud diameter	4.80 mm	
Stud height	1.80 mm	
Footprint, n studs	$8n - 0.2$ mm	$2 \times 4 = 15.8 \times 31.8$ mm
Wall thickness	1.50 mm	
Radial interference	≈ 0.10 mm	the clutch
Pull-off force / stud	8–15 N	MUST cite [SnapFitFEA1]/[SnapFitFEA2] envelope, §1.5.3
Brick mass (2×4)	≈ 2.5 g	robot dominates, not gravity

Brick library (8 SKUs): 1x1 1x2 1x4 1x6 2x2 2x3 2x4 2x6, standard height only. Plates excluded from v1 — they double the tiling search space for negligible research value.

Voxel grid: $8.0 \times 8.0 \times 9.6$ mm, integer (i, j, k), baseplate top at $k = 0$.

2.3.1 Collision geometry — hollow shell, MUST NOT be a solid box

A solid box forces a mated brick to sit 1.8 mm too high, resting on the studs. Model each brick as five convex boxes plus stud cylinders:

```

top plate   : box, 1.0 mm thick, at z = +4.3 mm
wall ±x     : box, 1.5 mm thick, cavity depth 8.6 mm
wall ±y     : box, 1.5 mm thick
studs (n)   : cylinder r = 2.4 mm, h = 1.8 mm, at z = +5.7 mm

```

2.3.2 Physics attributes that will otherwise waste a week

Parameter	Value	Why
contact_offset / rest_offset	0.3 mm / 0.1 mm	Defaults tuned for decimetre objects make 4.8 mm studs behave like mush
Physics dt (training)	1/240 s, 4 substeps	
Physics dt (evaluation)	1/500 s, 8 substeps	Below ~5 mm features, coarse stepping tunnels
Torsional friction	enabled	Without it a grasped brick spins about the finger normal
Solver iterations	24 position / 4 velocity	

Parameter	Value	Why
Contact filtering	exclude stud–cavity pairs of mated bricks	Residual contacts fight the joint model and inject energy
SDF mesh collision	enable on PhysX path only	The mechanism that made Factory's tight tolerances tractable

2.3.3 Joint state machine

Whether implemented by BrickSim or by a fallback kernel, the connection semantics are:



Gate conditions, all of which **MUST** hold between an active brick's anti-stud frame and a placed brick's stud frame:

Condition	Threshold
Lateral offset $\ \Delta_{xy}\ $	$< 1.2 \text{ mm}$
Vertical gap Δz	$\in [-0.3, +1.0] \text{ mm}$
Stud-axis misalignment	$< 4^\circ$
Yaw error to nearest 90°	$< 5^\circ$
Downward contact force along stud axis	$> 4 \text{ N}$
Dwell (continuous)	$> 40 \text{ ms}$

Two hard invariants:

- 1. Any applied joint wrench **MUST** be equal-and-opposite on both bodies. A one-sided force is a silent energy source that destabilizes tall structures and will be misdiagnosed as a policy failure in WP5.
- 2. The model **MUST** reproduce insertion/extraction asymmetry per [MechanicsSnapFit] — extraction force exceeds insertion force.

Alternative engagement detector worth testing: [DualArmSnapFit] detects snap engagement from **joint-velocity transients using proprioception alone** with >96% recall — cheaper than the force-threshold gate and hardware-transferable.

2.4 Platform specification

Hardware: Ubuntu 24.04, Ryzen 9 9950X3D, RTX 5090D (32 GB, Blackwell [sm_120](#)), 128 GB RAM.

Mandatory environment rules:

- 1. **Pin everything.** Isaac Lab tag [v3.0.0-beta2.patch1](#), Isaac Sim 6.0.1, specific Newton release, specific Warp release, specific cuRobo commit. Record hashes in [env.lock](#). **DO NOT** [git pull](#) on

develop mid-project — NVIDIA states the Newton integration is under heavy development with breaking changes and limited documentation expected, and no official support until release.

2. **Containerize.** Use the Isaac Lab Docker image as base. Isaac Lab apt dependencies have already broken this workstation's NVIDIA driver/library state once in conjunction with a separate MuJoCo installation. **DO NOT** install into the system Python.
3. **Keep the PhysX path alive.** Write the scene and task against Isaac Lab's backend-agnostic interfaces; select the engine by config flag.

Newton access pattern (verify against installed version). Isaac Lab exposes Newton's **Model**, **State**, **Control** and **Contacts** as live Warp arrays through `isaacsim_newton.physics.NewtonManager`, with a selection API (`get_attribute` / `set_attribute`) and `NewtonManager.add_model_change(...)` to signal modifications. This is the hook that allows a joint model to run as a GPU kernel across all environments with no per-env Python loop. Exact field names move between releases — **MUST** write a thin adapter (`sim/newton_compat.py`) so a rename costs one file.

Compute envelope (measure, do not assume). Budget **2048 envs state-only, 512–1024 with tiled cameras**. [VIRAL] used up to 64 GPUs for reliable vision distillation; treat any single-GPU vision estimate as optimistic.

2.5 Controller and motion stack

```

RL policy / scripted skill      20 Hz    → Δpose in EE frame (±2 mm / ±2°)
  ↓ policy-level action integrator (accumulate target, clip)  [IndustReal]
Task-space impedance control  240 Hz    → joint torques
  ↓
Newton solve                    240 Hz × 4 substeps

```

Use Isaac Lab's **OperationSpaceController** with null-space support rather than reimplementing. Gains:

```

K_p translational : diag(1200, 1200, 600) N/m      # deliberately soft in z
K_p rotational   : diag(40, 40, 40) N·m/rad
damping ratio    : 1.0
null-space       : posture task toward mid-range configuration

```

Compliance in z is what lets the brick find the socket instead of jamming.

Coarse motion (cuRobo). **MotionGen** for both arms with graph planning enabled; warm up kernels once at startup. World model contains table, baseplate, feeders, the growing structure (incrementally updated after each placement), and the other arm as a dynamic obstacle.

Dual-arm coordination — prioritized planning. The stabilizer plans first, because its brace pose is a task constraint, not a preference. Its final configuration is then added to the placer's world model as a static obstacle for the duration of the placement. The stabilizer is nearly static during insertion, which is exactly what makes prioritized decomposition valid here — **state this justification in the writeup**, it is a real argument rather than a shortcut.

Phase handoff. Coarse motion delivers the brick to a pre-insertion pose: directly above the target socket, +25 mm in z, with injected error sampled from the *measured* perception error distribution (or, if A3 shows vision is unnecessary, from the cuRobo placement error distribution). Matching the handoff distribution to the real error source is what makes the curriculum's top stage automatically correct.

2.6 Interface contracts

All modules read and write these. Schemas are normative.

```
// assembly_plan.json – output of M1, input to M5
{
  "structure_id": "stablelego_00417",
  "source": {"dataset": "StableLego", "index": 417},
  "voxel_origin": [0.0, 0.0, 0.75],
  "grid_pitch": [0.008, 0.008, 0.0096],
  "bricks": [
    { "id": "b_007", "type": "2x4", "grid_pos": [4,2,1], "yaw_index": 0,
    "color": "red" }
  ],
  "sequence": [
    {
      "step": 7,
      "brick_id": "b_007",
      "placer_arm": "B",
      "requires_brace": true,
      "brace": {
        "arm": "A",
        "target_brick_id": "b_005",
        "rationale": "weakest_joint_qp",          // brace at predicted
failure joint
        "predicted_failure_joint": ["b_003", "b_005"],
        "brace_pose": [0.12, -0.03, 0.81, 0, 0.7071, 0, 0.7071],
        "brace_force_N": 12.0,
        "brace_axis": [0, 0, 1],
        "expected_reaction_wrench": [0, 0, -22.4, 0.05, -0.02, 0.0],
        "end_effector": "allegro"
      },
      "grasp": { "face_pair": "long", "yaw_offset_deg": 0,
        "grip_width_m": 0.0148, "grip_force_N": 15.0 },
      "pre_insertion_pose": [0.032, 0.016, 0.7936, 0, 0, 0, 1],
      "target_pose": [0.032, 0.016, 0.7696, 0, 0, 0, 1],
      "mating_studs": [["b_003", 2], ["b_005", 2]]
    }
  ],
  "validation": {
    "stability_method": "stablelego_force_balance",
    "min_stability_score": 0.71,
    "weakest_joint": ["b_003", "b_005"],
    "drop_test_max_disp_mm": 0.42,
    "all_bricks_accessible": true,
    "brace_required_count": 4
  }
}
```

```
}
}
```

```
// insertion_episode.json – logged for EVERY attempt. This is the dataset.
{ "run_id": "A6_braced_seed03", "brick_id": "b_007",
  "policy": "residual_sparse_v2", "backend": "newton",
  "joint_model": "bricksim",
  "success": true, "steps": 48, "duration_s": 2.4,
  "initial_error_mm": 7.3, "initial_error_deg": 4.1,
  "final_error_mm": 0.21, "final_error_deg": 0.4,
  "peak_force_N": 21.7, "mean_force_N": 9.2, "impulse_Ns": 22.1,
  "force_budget_N": 30.0, "budget_violated": false,
  "brace_active": true, "brace_peak_reaction_N": 14.2, "brace_slipped":
false,
  "predicted_success_prob": 0.93,
  "snap_time_s": 1.9, "n_studs_engaged": 8, "failure_mode": null }
```

```
// joint_calibration.json – output of WP2
{ "reference": "analytic_yoshida_phase_diagram",
  "fitted": { "f_break_per_stud_N": 11.3, "insertion_peak_N": 8.9,
    "extraction_peak_N": 11.3, "asymmetry_ratio": 1.27,
    "lateral_stiffness_N_per_mm": 42.1,
    "gate_lateral_mm": 1.18, "ramp_time_s": 0.047 },
  "residual_rmse_N": 0.83,
  "phase_regime": "II", // per MechanicsSnapFit phase diagram
  "deformable_check_run": false }
```

```
// perception_result.json – output of M2 (only if WP6 proceeds)
{ "timestamp": 12.340, "camera": "wrist_B",
  "detections": [
    { "type": "2x4", "confidence": 0.94,
      "pose": [0.301, -0.248, 0.756, 0, 0, 0.0872, 0.9962],
      "pose_cov_diag": [4e-6, 4e-6, 1e-6, 1e-3, 1e-3, 4e-3],
      "graspable": true }
  ]}
```

2.7 Repository layout

```
brickassembly/
├─ env.lock          ★ pinned versions + hashes
├─ ledger.jsonl      ★ project ledger (§5.5)
├─ docker/           ★ derived from Isaac Lab image
├─ configs/
│   └─ scene.yaml  bricks.yaml  joint.yaml  control.yaml  rl.yaml
```

```
|   └─ structures/                                # selected StableLego indices + metadata
| └─ sim/
|   └─ brick_usd.py                             # programmatic USD generation per SKU
|   └─ scene.py                                 # backend-agnostic scene assembly
|   └─ newton_compat.py                         ★ thin adapter over Newton field names
|   └─ joint_model/
|     └─ bricksim_adapter.py                    ★ primary path
|     └─ warp_kernel.py                        ★ fallback / batched extension
|       └─ calibration.py                       # analytic fit, §1.5.3
|   └─ sensors.py
| └─ planner/
|   └─ voxelize.py  tiler.py
|   └─ stability.py                             # StableLego force-balance formulation
|   └─ accessibility.py                         # cuRobo batch IK + collision
|   └─ sequencer.py                            # physics-aware action masking
|   └─ bracing.py                              ★ weakest-joint brace assignment
| (contribution)
| └─ motion/
|   └─ curobo_planner.py  dual_arm.py  grasps.py
| └─ tasks/
|   └─ insertion_direct_env.py                 # Isaac Lab DirectRLEnv
|   └─ reward.py  curriculum.py  domain_rand.py  networks.py
|   └─ success_predictor.py                   # FORGE-style auxiliary head
| └─ perception/                             # CONDITIONAL on A3 outcome
| └─ orchestration/
|   └─ behavior_tree.py  skills.py  verify.py
| └─ eval/
|   └─ run_benchmark.py  ablations.py  stats.py  figures.py
| └─ tests/
| └─ scripts/
|   └─ 00_env_check.py                        ★ day-1 backend/VRAM/cuRobo benchmark
|   └─ 01_calibrate_joint.py
|   └─ 02_build_plan.py
|   └─ 03_train_insertion.py
|   └─ 04_full_assembly.py
|   └─ 05_run_experiments.py
```

★ marks files where project risk concentrates. A human SHOULD write these; agents MAY write the rest.

PART III — WORK PACKAGES

Each work package is a self-contained unit with explicit inputs, outputs, method, and machine-checkable acceptance criteria. An agent MUST NOT report a WP complete until every acceptance test passes and the result is written to the ledger.

Package summary

WP	Name	Depends on	Gate	Cut priority
WP0	Environment & platform verification	—	G0	never cut

WP	Name	Depends on	Gate	Cut priority
WP1	Joint mechanics integration	WP0	G1	never cut
WP2	Joint calibration	WP1	—	cut to literature values
WP3	Build planner + bracing assignment	WP1	G2	keep bracing, cut tiling
WP4	Motion stack + scripted baseline	WP1, WP3	G3 (critical)	never cut
WP5	RL insertion + vision decision	WP4	G4	keep residual only
WP6	Perception	WP5 (A3 result)	G5	first to cut
WP7	Full integration	WP5 (+WP6)	G6	never cut
WP8	Experiments & analysis	WP7	G7	never cut

WP0 — Environment & platform verification

Objective. Establish a pinned, containerized, benchmarked environment. Decide Newton vs PhysX empirically.

Inputs. Hardware per §2.4. Isaac Lab / Isaac Sim / Newton / Warp / cuRobo release identifiers.

Outputs. `env.lock`; `docker/`; `scripts/00_env_check.py` and its report; `sim/newton_compat.py`.

Method.

1. Build the Docker image from the Isaac Lab base. **DO NOT** install into system Python (§2.4 rule 2).
2. Pin every component; write `env.lock` with resolved hashes.
3. Run `00_env_check.py`, which MUST report:
 - steps/s on Newton and on PhysX for a 2-brick scene at 1, 512, 2048 envs
 - VRAM ceiling with and without tiled cameras
 - cuRobo MotionGen warmup time and per-plan latency
 - whether `NewtonManager` exposes per-env state as writable Warp arrays
4. Run `Isaac-Factory-PegInsert-Direct-v0` on both backends. **Read its source.** It is the template for WP5.
5. Write `newton_compat.py` wrapping every Newton field name the project touches.

Acceptance (Gate G0).

- ☐ Newton runs ≥ 2048 envs on a 2-brick scene at a measured, recorded throughput
- ☐ cuRobo plans in < 50 ms after warmup
- ☐ A Warp kernel can read and write per-env body state through `newton_compat.py`
- ☐ Impedance controller tracks a 50 mm free-space circle with < 0.5 mm RMS error
- ☐ `env.lock` exists and a fresh container rebuild reproduces the benchmark within 10%

On failure. Flip the backend flag to PhysX, enable SDF collision on studs and cavities, and record in the ledger that §1.6 contribution (2) is unavailable. This costs a contribution, not the project.

WP1 — Joint mechanics integration

Objective. Obtain a validated brick-joint model that batches to RL scale.

Inputs. WP0 environment. BrickSim repository.

Outputs. `sim/joint_model/bricksim_adapter.py`, or `warp_kernel.py` if the fallback triggers; `tests/test_joint.py` passing.

Method.

1. Clone and run BrickSim's own examples. Confirm the stability-prediction claim reproduces on their bundled cases.
2. **Batching probe (decisive).** Measure BrickSim's solve time as a function of parallel environment count. Record the scaling curve.
3. Branch:
 - **3a — BrickSim batches acceptably** (≥ 512 envs at usable throughput): write `bricksim_adapter.py` conforming to §2.3.3 semantics. WP1 is integration only. Record in the ledger that contribution (2) is *not* claimed.
 - **3b — BrickSim does not batch:** implement `warp_kernel.py` as a per-(env, connection) Warp kernel implementing the §2.3.3 state machine with equal-and-opposite wrenches, and **validate it against BrickSim** on single-environment cases. This is contribution (2). Record the fidelity comparison.
4. Either way, implement a hard-constraint variant for deterministic evaluation runs (Newton loop joints from a pre-allocated pool; runtime joint *creation* triggers a model rebuild and is too expensive).

Acceptance (Gate G1). `tests/test_joint.py` MUST pass:

- ☐ Gate fires inside tolerance and not outside — 6 boundary cases per gate condition in §2.3.3
- ☐ A mated pair holds 30 s under gravity with drift < 0.05 mm
- ☐ Release occurs at `f_break` $\pm 15\%$
- ☐ Extraction force $>$ insertion force (asymmetry, [MechanicsSnapFit])
- ☐ Two bricks cannot mate to the same stud
- ☐ **Energy test:** total system energy is non-increasing over 10,000 steps with no actuation $\leftarrow$ *the test that matters*
- ☐ Σ forces ≈ 0 across each mated pair (equal-and-opposite invariant)
- ☐ Kernel result identical at 1 env and at max envs
- ☐ A scripted top-down insertion snaps reliably from ≤ 2 mm initial error and holds 30 s

Failure mode to watch. "Tall structures are unstable" observed in WP5 is almost always a WP1 defect — a one-sided wrench or unfiltered mated contact injecting energy. The energy test catches it here, where it costs hours instead of days.

WP2 — Joint calibration

Objective. Ground the joint model's parameters in physics rather than hand-tuning. **Hard stop: 3 days.**

Inputs. WP1 joint model. [MechanicsSnapFit] phase diagram and closed-form relations. LEGO geometry §2.3.

Outputs. `joint_calibration.json` per §2.6; `sim/joint_model/calibration.py`.

Method.

1. Map the LEGO stud/cavity geometry onto the rigid-cylinder / thin-elastic-shell configuration of [MechanicsSnapFit]. Identify which of its four mechanical phases applies.
2. Derive insertion peak, extraction peak, asymmetry ratio and lateral stiffness from the closed-form relations.
3. Fit the joint model's (`f_break`, `ramp_time`, `gate tolerances`, `lateral stiffness`) to those values. Report residual RMSE.
4. Sanity-check magnitudes against the cantilever snap-fit FEA envelope (3–35 N insertion, 1.7–41 N retention) in [SnapFitFEA1]/[SnapFitFEA2].
5. **Optional, only if days remain:** VBD deformable thin-shell reference to test the analytic model's regime of validity. Not required.

Acceptance.

- ☐ `joint_calibration.json` exists with a stated phase regime and residual RMSE
- ☐ Fitted `f_break` lies within the literature envelope and is cited
- ☐ Asymmetry ratio > 1.0

On failure. Use literature values directly, record "literature-calibrated" in the ledger, and note the calibration gap as a limitation. **DO NOT** exceed the 3-day cap.

WP3 — Build planner and bracing assignment

Objective. Produce valid, buildable, brace-annotated assembly plans. **The bracing component is the project's primary contribution — do not treat it as a utility.**

Inputs. StableLego dataset. WP1 joint model. cuRobo from WP0.

Outputs. `assembly_plan.json` for the selected structure set; `planner/bracing.py`.

Method.

3.1 Structure selection. Sample five structures of graded difficulty from StableLego, with stability labels retained:

ID	Character	Difficulty introduced	Bricks (target)
S1	single column	sanity check	~6
S2	staggered wall	multi-brick layers, seam offset	~12
S3	spanning arch	unsupported span — single arm must fail	~18
S4	staircase	accessibility / gripper clearance	~24
S5	overhang + sub-assembly	full difficulty — single arm must fail	~31

MUST: verify by simulation that S3 and S5 genuinely fail single-arm before proceeding. Ablation A6 is the project's main experiment; if one arm succeeds anyway, the structures are wrong, not the result.

3.2 Layout. If starting from a mesh: voxelize at plate resolution, fill, collapse to brick layers by majority vote. Tile per layer with the 8-SKU library. Beam search, width 20, raster order, 10 restarts, objective:

```

J(T) = 10.0 · coverage(T)
      - 0.1 · |T|                # fewer, larger bricks
      + 2.0 · stagger_score(T, T_{k-1}) # seams must not align across
layers
      + 3.0 · connectivity_score(T, T_{k-1})
      - 5.0 · Σ_b inaccessible(b)    # cuRobo batch IK, §3.4

```

DO NOT claim the tiler as a contribution. Cite [Legolization], [KollskerLEGO3D].

3.3 Stability. Use the StableLego force-balance formulation. Retain both the binary stability verdict **and the weakest-joint location** — the latter is the input to bracing. Validate physically: instantiate the full structure with all joints mated, hold 2 s under gravity, apply a 0.5 N lateral impulse at the top, accept if max displacement < 1.0 mm.

3.4 Accessibility. For each candidate brick with placed set **P**: enumerate 4 grasp configurations (2 face pairs × 2 approach yaw); build the gripper swept volume from **z_target + 40 mm** down to **z_target**; **batch-query cuRobo** for all configurations × all candidate bricks in the layer in a single GPU call against a world model containing **P**. Update the world model incrementally after each placement. Cite [BricksToBots].

3.5 Sequencing. Precedence graph: **b_j** precedes **b_i** if **b_j** supports **b_i**, or if **b_i**'s top-down insertion corridor passes through **b_j**. Order using the physics-aware action-mask formulation of [PhysASP], or a simpler greedy order with a recorded runtime justification.

3.6 Bracing assignment — the contribution. For each brick **b_i** in sequence order:

1. Compute the expected insertion wrench: **n_studs × f_insert** along the stud axis, plus a lateral uncertainty term (±3 N).
2. Query the joint model's internal force distribution (BrickSim QP, or the fallback's equivalent) for the current partial structure under that wrench.
3. Identify the joint with the smallest margin to **f_break** — the **predicted failure joint**.
4. If margin < safety factor (default 1.5), set **requires_brace = true** and compute a brace pose on the brick adjacent to that joint, with a brace force opposing the predicted failure direction.
5. Emit **expected_reaction_wrench** into the plan so WP7 can compare predicted vs. measured.

Baselines that MUST also be implemented, because A6/A9 compare against them:

- **brace_none** — no stabilizer
- **brace_nearest** — brace the nearest structurally-connected placed brick (the naive heuristic)
- **brace_weakest_joint** — the proposed method

Acceptance (Gate G2).

- ☐ All five structures produce plans passing the drop test with **all_bricks_accessible: true**
- ☐ S3 and S5 have **brace_required_count > 0** and are **verified to fail single-arm**
- ☐ **tests/test_planner.py**: tiling covers 100% of each voxel model; no grid overlaps; every sequenced brick supported by already-placed bricks or baseplate; topological order valid; **requires_brace** flags match the force-margin analysis
- ☐ All three bracing strategies emit valid plans for the same structures

On failure. If S4/S5 fail accessibility, either swap to a simpler StableLego sample **or** switch the placer to a clutch tool (a wrist-mounted anti-stud plate with an ejector pin — how real automated LEGO handling works, giving perfect top-down clearance). **DO NOT** redesign the hand.

WP4 — Motion stack and scripted end-to-end baseline

Objective. A complete, working pipeline with ground-truth poses and a scripted inserter. **This is the schedule's load-bearing wall.**

Inputs. WP1 joint model, WP3 plans, cuRobo.

Outputs. *motion/*, *orchestration/*, a scripted insertion skill, benchmark run on S1–S5.

Method.

1. *grasps.py*: 4 antipodal grasps per brick, scored by $\text{clearance_margin_mm} - 2.0 \cdot |\text{offset_from_centroid_mm}| + 5.0 \cdot \text{aligned_with_place_orientation}$. Fingertips centred at 4.8 mm above brick base; grip width = footprint – 1.0 mm; grip force 15 N. Validate each with a 1 s shake test (± 0.05 m at 3 Hz); prune failures permanently.
2. *curobo_planner.py*: MotionGen wrappers with incremental world-model updates.
3. *dual_arm.py*: prioritized planning, stabilizer first (§2.5). On plan failure: retreat the stabilizer to a secondary brace pose on the same support brick and retry; then serialize; then escalate to recovery.
4. Scripted insertion skill: descend at 5 mm/s; on contact, 2 mm-radius Archimedean spiral search at 3 mm/s holding 8 N down, with $\pm 1.5^\circ$ wiggle about x/y at 2 Hz. **This becomes the residual base controller in WP5** — write it to be reused.
5. *behavior_tree.py* with *py_trees*. **DO NOT** hand-roll a state machine; the retry logic will be needed.

```
Sequence: AssembleStructure
├─ Repeat(each step)
│   └─ Selector: PlaceBrick [≤3 retries]
│       ├── Sequence: NominalPlacement
│       │   ├── [A] MoveToBracePose + ApplyBraceForce (if
requires_brace)
│       │   ├── [B] Perceive → Grasp → TransportToPreInsertion
│       │   ├── [B] RunInsertionSkill (timeout 10 s)
│       │   ├── VerifySnap
│       │   ├── [B] ReleaseAndRetract
│       │   └─ [A] ReleaseBrace
│       └─ Sequence: Recovery
│           ├── LogFailure → [B] LiftAndRegrasp
│           └─ [B] ScriptedInsertionFallback
```

The stabilizer brace **precedes the placer's transport**, not just its insertion, so cuRobo plans the placer around a settled arm.

Acceptance (Gate G3 — critical). With ground-truth poses and the scripted inserter, over $n = 20$ trials each:

- ☐ S1, S2 complete $\geq 90\%$

- ☐ S3 completes $\geq 60\%$ **with bracing active**
- ☐ Every module's JSON contract in §2.6 is exercised end-to-end
- ☐ `tests/test_control.py` passes: impedance step response $< 5\%$ overshoot; null-space posture does not perturb task space (< 0.1 mm); F/T sensor within 5% of a known static load

On failure. STOP ALL FORWARD WORK. A complete pipeline is worth more than a policy with nothing to plug into. Cut S4/S5 to "extended structures," reduce the brick library, simplify the grasp set — but reach G3.

WP5 — RL insertion and the vision decision

Objective. A precision insertion policy, and an empirical answer to whether vision is needed.

Inputs. WP4 pipeline and scripted controller. Factory/FORGE source read in WP0.

Outputs. Trained policies; A1/A3/A4 ablation results; the vision decision.

Method — run order is normative.

5.1 Environment. `tasks/insertion_direct_env.py` as an Isaac Lab `DirectRLEnv`. Enable CUDA-graphing. Verify throughput before training anything.

5.2 MDP. Episode starts at the pre-insertion pose with a grasped brick and the stabilizer already bracing; ends on snap success, force violation, drop, or timeout (200 steps @ 20 Hz).

Actor observation (deployed configuration):

Group	Dim	Contents
Proprioception	24	$\mathbf{q}(7)$, $\dot{\mathbf{q}}(7)$, EE position(3), EE rotation 6D(6), gripper width(1)
F/T history	60	last 10 steps $\times$ 6D wrench, EE frame, normalized by 30 N / 2 N·m
Visual embedding	64	conditional — CNN over 128 $\times$ 128 RGB-D wrist crop
Task context	21	Δ pose target $\rightarrow$ current (3+6), brick type one-hot(8), engagement scalar(1), n_studs(1), reserved(2)
Brace state	7	stabilizer contact wrench (6) + brace-active flag (1)
Force budget	1	max allowable force, per [FORGE]
Previous action	6	
Total	183	(119 without vision)

Critic privileges (+26) MUST be chosen per [InformedAAC] to be **student-inferable from force history** — per-stud engagement depth and contact count qualify; exact brick-to-socket relative pose during occlusion does not. Record the chosen set and the justification.

Action: 6-D $[\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}]$ in EE frame, `tanh`-squashed to ± 2 mm / $\pm 2^\circ$, integrated by the policy-level action integrator. Optional 9-D variant adds per-axis translational stiffness in [0.3, 2.0] — ablation A8 only.

5.3 Run order (normative).

Run	What	Purpose	Est.
R1	Residual + sparse reward — RL residual on the WP4 scripted controller, sparse goal-reaching reward only	PRIMARY. [Marougkas]	~3 h
R2	Residual + dense reward	isolates reward shaping's contribution (A-dense)	~3 h
R3	End-to-end + dense reward, privileged state	the conventional baseline (A1, A4)	~4 h
R4	R1 with vision group zeroed	ablation A3 — the vision decision	~3 h
R5	Vision student distilled from R1/R3	only if R4 underperforms	~6 h+

If R3 does not reach > 85% success at curriculum stage 3, the defect is in the reward or the joint model, not the RL. Stop and fix WP1/WP2.

5.4 Curriculum. Advance when success rate over the last 100 episodes > 80%; regress below 50%.

Stage	Pos error	Rot error	Socket occupancy	Bricks
0	U(0, 2 mm)	U(0, 2°)	isolated	2x4
1	U(0, 5 mm)	U(0, 5°)	isolated	2x4, 2x2
2	U(0, 10 mm)	U(0, 8°)	1 neighbour	all 2xN
3	U(0, 15 mm)	U(0, 12°)	2–3 neighbours	all 8
4	measured handoff error (§2.5)	—	≤ 4 neighbours	all 8

5.5 Domain randomization (per episode).

Parameter	Range
Brick–brick friction	U(0.3, 0.9)
Brick–finger friction	U(0.5, 1.2)
f_break per stud	U(6, 16) N — narrow to the WP2-calibrated range once available
Gate lateral tolerance	U(0.9, 1.5) mm
Brick mass	±15%
Baseplate pose	±1 mm, ±0.5°
Impedance gain multiplier	U(0.8, 1.25)
Brace force	U(8, 18) N, 5% chance of brace slip
F/T noise σ / bias	0.15 N / 0.01 N·m; bias random-walk 0.02 N/s

Parameter	Range
Control latency	0–2 steps
Force budget	U(20, 45) N — [FORGE] conditioning

The brace-slip randomization earns its place: it forces the placer to be robust to an imperfect stabilizer, which is the realistic case and a good talking point.

5.6 Success predictor. Train the [FORGE]-style auxiliary head on episode outcomes. It replaces the hand-coded verification threshold in WP7 and enables early termination.

Acceptance (Gate G4).

- ☐ A policy reaching $\geq 85\%$ insertion success at stage 3 (15 mm / 12° initial error) with mean peak force < 30 N
- ☐ R1 vs R3 comparison recorded (A1, A4)
- ☐ **R4 result recorded, and the WP6 decision made and written to the ledger**
- ☐ Success predictor AUC > 0.85 on held-out episodes
- ☐ `tests/test_env.py`: obs dim matches spec; no NaNs over 10k random-action steps; reward bounded; all terminal conditions reachable; Newton and PhysX agree on 100 steps to $< 1e-3$

On failure. Ship R1 (residual) as the main result and report end-to-end RL as an underperforming ablation. That is honest and consistent with [Marougkas].

WP6 — Perception (CONDITIONAL)

Execute only if R4 (no-vision) underperforms R1 by a margin that matters. If force-only matches multimodal, **skip this entire work package**, report the negative result, and spend the time on WP8. That is a publishable finding in its own right (§1.6 item 4).

Method (if executed).

- Generate 20k synthetic renders via Isaac Lab tiled cameras with full randomization: brick count (5–30), types, colours, tray poses, three lights (position, intensity 0.5–2.0, colour temperature), table/tray materials, camera extrinsics (± 10 mm, $\pm 3^\circ$), exposure, 0–5 distractor primitives.
- Fine-tune YOLOv8s-seg (640×640, COCO init). Target mAP50 > 0.9 .
- Pose: mask $\rightarrow$ depth $\rightarrow$ point cloud $\rightarrow$ centroid + top-surface PCA $\rightarrow$ point-to-plane ICP against the known brick mesh; snap yaw to the nearest of 4 symmetric solutions.
- Characterize and record the error distribution.** That distribution is the curriculum's stage-4 spec (§2.5). Measure it; do not guess it.
- Fine stage: 128×128 RGB-D wrist crop $\rightarrow$ 4-layer CNN (32/64/128/256, stride 2, GroupNorm, SiLU) $\rightarrow$ 64-d. ~ 0.4 M params — this runs inside the RL loop.
- Distill via `rs1_rl` student–teacher, applying the [InformedAAC] privilege constraint from §5.2.

Acceptance (Gate G5). Perception-driven pipeline completes S1–S3 at $\geq 70\%$; pose error distribution documented.

Compute warning. [VIRAL] needed up to 64 GPUs for reliable vision distillation. Budget accordingly: reduce crop to 96×96 and envs to 512 before concluding the method fails.

WP7 — Full integration

Objective. The complete system running end-to-end on all structures, with all three bracing strategies.

Method.

1. Swap the scripted inserter for the WP5 policy inside the behaviour tree, retaining the scripted fallback on the third retry. "RL-first with scripted fallback" is the system being reported.
2. Replace threshold verification with the WP5 success predictor.
3. Wire all three bracing strategies as a runtime configuration.
4. Log `insertion_episode.json` (§2.6) for **every** attempt — including the brace reaction force. This log is the dataset for WP8.

Acceptance (Gate G6).

- ☐ Full loop runs on S1–S5 for all three bracing strategies without manual intervention
- ☐ Predicted vs. measured brace reaction wrench recorded per placement
- ☐ Verification: 200 ms after release, all expected studs **MATED**, brick pose error < 0.5 mm / 1°, velocity < 1 mm/s

WP8 — Experiments and analysis

See Part IV.

PART IV — EVALUATION PROTOCOL

4.1 Primary metrics

Metric	Definition	Target
Per-brick insertion success	verified placements / attempts (incl. retries)	≥ 90%
First-attempt success	verified without retry	≥ 80%
Structure completion rate	fully assembled and stable / trials	≥ 70% (S1–S3)
Mean peak insertion force	per successful insertion	< 30 N
Force impulse	$\int F dt$ per insertion	reported
Mean insertion time	pre-insertion pose → verified snap	< 4 s
Cycle time	per brick, pick to verified	< 15 s
Final RMS brick pose error	vs. plan	< 0.5 mm
Post-assembly robustness	max displacement under 1 N lateral impulse	< 2 mm
Brace reaction force	measured, per supported placement	reported vs. predicted

Metric	Definition	Target
Brace prediction error	$\ \text{measured} - \text{expected_reaction_wrench} \ $	reported

4.2 Statistical protocol

MUST: report mean $\pm$ 95% CI over 20 seeds. Single runs are not results. Full benchmark = 5 structures $\times$ 20 seeds $\times$ 5 configurations = 500 assembly trials.

For the primary comparison (A6), use a paired test across matched seeds and report effect size, not only significance. The benchmark to beat is [DualArmSnapFit]'s **30% reduction in peak impact force**.

4.3 Ablations

#	Ablation	Question	Status
A6	brace_weakest_joint vs brace_nearest vs brace_none on S3/S5	Does principled bracing improve insertion success and reduce peak force?	MAIN EXPERIMENT
A3	no vision (F/T + proprio only)	Is vision necessary at sub-mm clearance under self-occlusion?	Run in WP5, gates WP6
A10	calibrated joint model vs hand-tuned	Does joint-model fidelity change learned behaviour?	Contribution (3)
A1	RL vs scripted spiral search	Does RL earn its place?	supporting
A4	residual vs end-to-end	Does the model-based prior help or constrain?	supporting
A-dense	sparse vs dense reward (R1 vs R2)	Is dense shaping necessary?	supporting, tests [Maroungkas]
A2	no F/T (vision only)	Is force sensing necessary?	supporting
A5	no curriculum	Quantify the curriculum	supporting
A7	no DR, evaluated on held-out physics	Robustness	supporting
A8	9-D learned stiffness vs 6-D fixed	Is variable impedance worth it?	optional
A9	Allegro stabilizer vs parallel-jaw stabilizer	Does the hand earn its place?	optional

Cut order if time runs short: A8, A9, A5, A7, A2 — in that order. **Never cut A6 or A3.**

4.4 What each result means

- A6 positive** → the contribution holds. Report effect size against [DualArmSnapFit]'s 30% benchmark and against [BUDS]'s 56% unstructured-baseline improvement.
- A6 null** → the honest finding is that bracing placement does not matter for interlocking joints, which contradicts the intuition from masonry ([LightVault]) and is itself informative — but the title must

change.

- **A3 null (vision unnecessary)** → system simplification, WP6 skipped, publishable negative result.
- **A10 positive** → implications beyond LEGO for how much simulation fidelity contact-rich RL requires.
- **A10 null** → a useful statement that cheap joint models suffice, which strengthens the case for BrickSim-style abstractions.

PART V — AI ORCHESTRATION PIPELINE

This part specifies how the project is executed by a mixed human/agent team.

5.1 Agent roles

Role	Responsibility	May write	Must not
Orchestrator	Schedules WPs, checks gates, decides cuts, owns the ledger	<code>ledger.jsonl</code>	Write implementation code
Researcher	Reads cited sources before implementation; produces design notes	<code>docs/</code>	Change locked decisions (§2.2) without a ledger entry
Implementer	Writes module code against §2.6 contracts	<code>planner/</code> , <code>motion/</code> , <code>tasks/</code> , <code>perception/</code> , <code>eval/</code>	Touch ★ files; change <code>env.lock</code>
Verifier	Writes and runs acceptance tests; independent of the Implementer	<code>tests/</code>	Modify implementation to make a test pass
Analyst	Runs experiments, produces statistics and figures	<code>eval/</code> , <code>results/</code>	Select seeds post hoc
Human (Sakib)	Owens ★ files, all gate decisions, all scope cuts	anything	—

Critical separation. The Implementer MUST NOT write its own acceptance tests. The Verifier writes tests from this document's acceptance criteria, not from the implementation. This is the only defence against an agent that makes a test pass by weakening it.

5.2 Task graph

```
# tasks.yaml — machine-readable. The orchestrator reads this.
project: brickassembly
version: 3.0
gates_are_blocking: true

work_packages:
- id: WP0
  name: environment_verification
```

```

depends_on: []
gate: G0
owner: human
cuttable: false
acceptance_ref: "$WP0"
on_fail: {action: switch_backend, to: physx, record:
"contribution_2_unavailable"}

- id: WP1
  name: joint_mechanics
  depends_on: [WP0]
  gate: G1
  owner: human
  cuttable: false
  branch:
    probe: bricksim_batching_scaling
    if_batches: {path: adapter, contribution_2: false}
    else:      {path: warp_kernel, contribution_2: true}

- id: WP2
  name: joint_calibration
  depends_on: [WP1]
  gate: null
  owner: implementer
  hard_cap_days: 3
  cuttable: true
  on_cut: {fallback: literature_values, record:
"calibration_gap_limitation"}

- id: WP3
  name: planner_and_bracing
  depends_on: [WP1]
  gate: G2
  owner: implementer
  cuttable: partial
  must_keep: [bracing.py, three_bracing_strategies]
  preflight:
    - "verify S3 and S5 fail single-arm"

- id: WP4
  name: motion_and_scripted_baseline
  depends_on: [WP1, WP3]
  gate: G3
  gate_criticality: blocking_all_forward_work
  owner: implementer
  cuttable: false

- id: WP5
  name: rl_insertion
  depends_on: [WP4]
  gate: G4
  owner: implementer
  run_order: [R1, R2, R3, R4]      # normative; R5 conditional
  decision_point:

```

```

    id: vision_decision
    input: "A3 result (R4 vs R1)"
    outputs: [proceed_WP6, skip_WP6]

- id: WP6
  name: perception
  depends_on: [WP5]
  conditional_on: {decision: vision_decision, value: proceed_WP6}
  gate: G5
  owner: implementer
  cuttable: true
  cut_priority: 1                # first to cut

- id: WP7
  name: integration
  depends_on: [WP5]
  optional_depends_on: [WP6]
  gate: G6
  owner: implementer
  cuttable: false

- id: WP8
  name: experiments_and_analysis
  depends_on: [WP7]
  gate: G7
  owner: analyst
  cuttable: false
  must_run: [A6, A3, A10]
  cut_order: [A8, A9, A5, A7, A2]

```

5.3 Task card template

Every unit of agent work MUST be issued as a card in this form. Cards without acceptance criteria MUST be rejected by the agent.

```

task_id: WP3-04
work_package: WP3
title: Implement weakest-joint bracing assignment
role: implementer
context_files:
  - master_report.md §1.6, §2.6, §WP3.6
  - sim/joint_model/bricksim_adapter.py
  - planner/stability.py
inputs:
  - assembly_plan.json (without brace fields)
  - joint model force-distribution query API
outputs:
  - planner/bracing.py
  - assembly_plan.json (brace fields populated)
method_ref: "§WP3.6 steps 1-5"
acceptance:

```

```

- "three strategies implemented: none / nearest / weakest_joint"
- "expected_reaction_wrench emitted for every braced placement"
- "requires_brace flags match force-margin analysis at safety factor 1.5"
- "tests/test_bracing.py passes (written independently by verifier)"
forbidden:
- "modifying the joint model"
- "changing the safety factor without a ledger entry"
escalate_if:
- "force-distribution query is unavailable or slower than 50 ms"
definition_of_done: "all acceptance items green AND ledger entry written"

```

5.4 Standing guardrails for all agents

These apply to every task card and MUST be included in every agent's system context.

1. **Check §1.2 before writing any module.** Four components already exist as published open-source work. Rebuilding them is the project's dominant failure mode.
2. **Never modify `env`. Lock or pull on `develop`.** Version drift on a beta platform destroys reproducibility (§2.4).
3. **Never weaken an acceptance test to make it pass.** Escalate instead.
4. **Never claim a contribution that §1.2 shows is published.** Specifically: gripper-accessibility coupling, generative brick design, stability heuristics, sequencing.
5. **Every fabricated number must trace to a source.** Force magnitudes to [SnapFitFEA1]/[SnapFitFEA2]; geometry to §2.3; behavioural claims to §7.4.
6. **Write to the ledger on every gate, decision, cut, and failure.** An undocumented decision is a lost decision.
7. **If a cited source contradicts this document, the source wins.** Escalate to the human and record it.
8. **Do not tune on the evaluation set.** Curriculum and hyperparameter selection use development structures only.

5.5 Project ledger

Append-only JSONL. The orchestrator owns it. It is what makes the project reconstructible.

```

{"ts":"2026-09-16T09:12:00Z","type":"gate","id":"G0","result":"pass",
 "evidence":{"newton_steps_per_s_2048env":74200,"curobo_plan_ms":31,
  "vram_peak_gb":19.4,"warp_state_writable":true},
 "decision":"proceed with Newton backend"}

{"ts":"2026-09-17T14:40:00Z","type":"branch","wp":"WP1",
 "probe":"bricksim_batching_scaling",
 "measurement":{"envs":[1,64,512,2048],"solve_ms":[5.1,22,180,"OOM"]},
 "decision":"warp_kernel path","implication":"contribution_2 claimed",
 "validation_plan":"fidelity comparison vs BrickSim on 150 single-env
cases"}

{"ts":"2026-09-30T11:05:00Z","type":"decision","id":"vision_decision",
 "input":
 {"R1_success":0.88,"R4_success":0.86,"delta":0.02,"ci_overlap":true},

```

```
"outcome": "skip_WP6",
"rationale": "force-only matches multimodal; consistent with MarkertDQN, SymmetrySoftWrist",
"reallocation": "WP6 time → A6 seed count 20→40"}

{"ts": "2026-10-02T16:20:00Z", "type": "cut", "target": "A8",
"reason": "schedule", "authorised_by": "human"}
```

Required entry types: **gate**, **branch**, **decision**, **cut**, **failure**, **deviation**, **result**.

5.6 Reading order for a cold-start agent

An agent joining with no context MUST load, in this order:

- 1. §0.3 (the single most important instruction) and §1.2 (do not rebuild)
- 2. §2.2 (locked decisions) — these are not open for re-litigation
- 3. §2.6 (interface contracts) — the module's inputs and outputs
- 4. The specific WP section for its task
- 5. §5.4 (standing guardrails)
- 6. **ledger.jsonl** — what has already been decided and measured

Do not load the full literature review unless the task is a writeup task. §1.2 and §2.2 carry the operational content.

5.7 Schedule shape

The original 7-week schedule assumed building the joint model, planner, and sequencer from scratch. §1.2 removes roughly two weeks of build work and returns it as experiment time.

Phase	Weeks	Contents	Gate
Foundation	1	WP0, WP1	G0, G1
Planning	2	WP2 (capped), WP3	G2
Baseline	3	WP4	G3 — blocking
Learning	4–5	WP5 (R1 → R4), vision decision	G4
Conditional	6	WP6 or expanded experiments	G5
Results	7	WP7, WP8, writeup	G6, G7

If G3 is missed, everything after shifts and WP6 is cut automatically.

PART VI — RISK REGISTER

#	Risk	P	Impact	Mitigation	Trigger point
R1	Isaac Lab Newton beta breaks or lacks a needed feature	High	High	Pinned versions, Docker, <code>newton_compat.py</code> , PhysX fallback by flag	WP0, then continuous
R2	Joint model injects energy; tall structures unstable	Med	Critical	Equal-and-opposite invariant; energy unit test in WP1	WP1
R3	BrickSim does not batch	Med	Med → opportunity	Becomes contribution (2); Warp kernel validated against it	WP1 probe
R4	RL fails to learn insertion	Med	High	R1 (residual) is primary; R3 gates reward design before vision	WP5 R3
R5	S3/S5 do not actually fail single-arm	Med	High	Preflight verification in WP3; redesign structures, not the result	WP3 preflight
R6	32 GB VRAM caps envs below useful throughput	Med	Med	Measured in WP0; reduce crop to 96×96, envs to 512	WP0
R7	Vision student ≪ state teacher (teacher unrealizability)	Med	Med	[InformedAAC] privilege constraint; A3 may make it moot	WP5 R4/R5
R8	Franka+Allegro USD composition consumes days	Med	Med	1-day cap → switch stabilizer to stock KUKA+Allegro	WP0
R9	Isaac Lab install collides with existing MuJoCo environment	Med	High	Docker. Has already happened once on this machine.	WP0
R10	Scope creep back toward the original spec	High	High	§1.2 and §2.2. Re-read at every gate.	weekly
R11	Agent claims a published result as a contribution	Med	High (reputational)	Guardrail §5.4.4; Researcher role reviews all claims before writeup	WP8
R12	Deliverable ambiguity (research vs. course project)	Med	High	§0.5 — resolve with instructor before WP0	before WP0

R2, R10 and R11 are the three that actually damage projects of this shape. R2 wastes weeks; R10 wastes the schedule; R11 wastes the result.

6.1 Contingency tree

```
G0 fail → PhysX backend + SDF collisions; drop contribution (2); continue
G1 fail → STOP. Joint model is foundational. Fix before anything else.
G2 fail → swap StableLego samples OR switch placer to clutch tool; do not
redesign hand
G3 fail → STOP ALL FORWARD WORK. Cut structures/library/grasps until G3
passes.
G4 fail → ship R1 (residual) as main result; report end-to-end as failed
ablation
G5 n/a → if A3 null, skip WP6 entirely and reallocate time to A6 seeds
G6 fail → report per-module results without full-loop completion rates
G7      → if A6 null, retile the work around the negative result and
A3/A10
```

PART VII — APPENDICES

7.1 Reward specification

Primary (R1, sparse — per [Marougkas]):

```
r = +50.0 if snap_verified          # terminal
    -10.0 if force_violation or drop or timeout # terminal
    0.0  otherwise
```

The scripted base controller carries the shaping burden. The residual learns only the correction.

Dense variant (R2/R3, retained for ablation A-dense):

```
r_dist   = 1.0 * exp(-||p_err|| / 0.005)      # σ = 5 mm
r_align  = 0.3 * (1.0 - angle_err / π)
r_engage = 0.5 * (engagement_depth / 1.8mm)    # drives studs INTO the
cavity
r_force  = -0.5 * max(0, ||F|| - F_budget) / 35.0 # F_budget is an
observation, [FORGE]
r_action = -0.01 * ||a||²
r_time   = -0.005
r_snap   = +50.0 (terminal)
r_fail   = -10.0 (terminal)
```

r_engage is the term that matters. A pure distance reward plateaus at "hovering correctly above the socket" — the policy learns to reach the right pose and stop, because the remaining reward gradient is flat until contact.

7.2 PPO hyperparameters

```

num_envs          2048 (state) / 512-1024 (vision)  ← measured in WP0, not
assumed
rollout_len       32                               gamma          0.99
minibatch         16384                             gae_lambda         0.95
epochs            4                                 clip_eps           0.2
lr                3e-4 cosine                       entropy_coef       0.002 → 0.0005 (linear)
value_coef        0.5                               max_grad_norm      1.0
network           MLP [512,256,128] ELU, separate actor/critic trunks
total steps       120M (R1-R4), 200M (R5)
normalization     running mean-std on obs and reward
CUDA graphs       enabled

```

Always log: per-stage success rate, mean peak force, insertion time, action-magnitude distribution, force-budget violation rate. If peak force creeps toward the budget while success rises, the policy has found slam-and-snap; raise the force-budget conditioning weight rather than the penalty.

7.3 Failure-mode reference

Symptom	Likely cause	Fix
Policy hovers, never contacts	r_dist saturates pre-contact; force penalty too harsh	raise r_engage ; penalize only above budget
Success at the force cap	slam-and-snap exploit	strengthen budget conditioning; add jerk penalty
Learns stages 0–1, collapses at 2	neighbour bricks genuinely harder; curriculum jump too large	insert an intermediate stage; raise min-stage dwell
Structures explode / energy gain	one-sided joint wrench, or unfiltered mated contacts	equal-and-opposite invariant; contact filtering (§2.3.2)
Bricks tunnel through studs	substep count too low for 4.8 mm features	8 substeps; reduce contact_offset
Vision student ≪ teacher	teacher unrealizability ([StudentInformedTeacher])	constrain critic privileges per [InformedAAC]; or A3 makes it moot
Brick squirts out of gripper	torsional friction missing; grip force low	enable torsional friction; raise to 20 N
Newton API breaks after update	tracked develop	git checkout the pinned tag; see guardrail §5.4.2

Symptom	Likely cause	Fix
Bracing shows no effect	S3/S5 don't actually require bracing	WP3 preflight failed; redesign structures

7.4 Bibliography

Keys are used throughout this document.

Brick-specific simulation, stability and planning

- **[BrickSim]** Wen, H., et al. (2026). *BrickSim: A Physics-Based Simulator for Manipulating Interlocking Brick Assemblies*. arXiv. [10.48550/arXiv.2603.16853](#) · [github.com/intelligent-control-lab/BrickSim](#)
- **[StableLego]** Liu, R., Deng, K., Wang, Z., & Liu, C. (2024). *StableLego: Stability Analysis of Block Stacking Assembly*. IEEE RA-L 9(11), 9383–9390. [10.1109/LRA.2024.3414281](#)
- **[PhysASP]** Liu, R., et al. (2024). *Physics-Aware Combinatorial Assembly Sequence Planning Using Data-Free Action Masking*. IEEE RA-L. [10.1109/LRA.2025.3555074](#)
- **[BricksToBots]** Barghi, A., et al. (2024). *From Bricks to Bots: Automated Collision-Aware Sequence Planning for LEGO Reconstruction with a Two-Finger Gripper*. [10.1109/ICCIA65044.2024.10768173](#)
- **[Legolization]** Luo, S.-J., et al. (2015). *Legolization: Optimizing LEGO Designs*. ACM TOG 34(6), Art. 222. [10.1145/2816795.2818091](#)
- **[Testuz]** Testuz, R., Schwartzburg, Y., & Pauly, M. (2013). *Automatic Generation of Constructable Brick Sculptures*. Eurographics. [10.2312/conf/EG2013/short/081-084](#)
- **[KollskerLEGO3D]** Kollsker, T., et al. (2021). *Optimisation and Static Equilibrium of Three-Dimensional LEGO Constructions*. [10.1007/s43069-021-00062-3](#)
- **[KollskerLEGO2D]** Kollsker, T., et al. (2021). *Models and Algorithms for Optimising Two-Dimensional LEGO Constructions*. EJOR. [10.1016/j.ejor.2020.07.004](#)
- **[ASPGraph]** Ma, et al. (2022). *Planning Assembly Sequence with Graph Transformer*. ICRA. [10.1109/ICRA48891.2023.10160424](#)
- **[BudgetBrick]** Ahn, et al. (2022). *Budget-Aware Sequential Brick Assembly with Efficient Constraint Satisfaction*. TMLR. [10.48550/arXiv.2210.01021](#)

Generative brick design (all superseding the deleted stretch goals)

- **[BrickGPT]** Pun, et al. (2025). *Generating Physically Stable and Buildable Brick Structures from Text*. ICCV. [10.1109/ICCV51701.2025.01373](#) / [10.48550/arXiv.2505.05469](#)
- **[RollbackFree]** (2026). *Rollback-Free Stable Brick Structures Generation*. [10.48550/arXiv.2605.06947](#)
- **[BrickAnything]** (2026). *BrickAnything: Geometry-Conditioned Buildable Brick Generation*. [10.48550/arXiv.2605.26182](#)
- **[BrickNet]** (2026). *BrickNet: Graph-Backed Generative Brick Assembly*. [10.48550/arXiv.2604.22984](#)
- **[LegoACE]** (2025). *LegoACE: Autoregressive Construction Engine for Expressive LEGO Assemblies*. SIGGRAPH Asia. [10.1145/3757377.3763881](#)

Contact-rich RL assembly

- **[IndustReal]** Tang, B., et al. (2023). *IndustReal: Transferring Contact-Rich Assembly Tasks from Simulation to Reality*. RSS. [10.48550/arXiv.2305.17110](#)
- **[FORGE]** Noseworthy, M., et al. (2025). *FORGE: Force-Guided Exploration for Robust Contact-Rich Manipulation under Uncertainty*. IEEE RA-L. [10.1109/LRA.2025.3551637](#)
- **[Marougkas]** Marougkas, I., et al. (2025). *Integrating Model-Based Control and RL for Sim2Real Transfer of Tight Insertion Policies*. ICRA. [10.1109/ICRA55743.2025.11128860](#)
- **[SnapCurriculum]** Monnet, et al. (2025). *Leveraging Reinforcement and Curriculum Learning for Flexible Robot-Based Snap-Fit Assembly Automation*. at-Automatisierungstechnik. [10.1515/auto-2024-0177](#)
- **[VarCompliance]** Beltran-Hernandez, C., et al. (2020). *Variable Compliance Control for Robotic Peg-in-Hole Assembly*. Applied Sciences 10(19). [10.3390/app10196923](#)
- **[MultiModalImpedance]** Chen, et al. (2023). *Multimodality Driven Impedance-Based Sim2Real Transfer Learning for Robotic Multiple Peg-in-Hole Assembly*. IEEE T-Cyb. [10.1109/TCYB.2023.3310505](#)
- **[AdmittanceResidual]** Zhang, et al. (2023). *Efficient Sim-to-Real Transfer with Online Admittance Residual Learning*. [10.48550/arXiv.2310.10509](#)
- **[FuzzyVTS]** Hou, et al. (2022). *Fuzzy Logic-Driven Variable Time-Scale Prediction-Based RL for Multiple Peg-in-Hole*. IEEE T-ASE. [10.1109/TASE.2020.3024725](#)

Force vs. vision (the contested question, §1.4.1)

- **[MarkertDQN]** Markert, et al. (2023). *Robotic Peg-in-Hole Insertion with Tight Clearances: A Force-Based Deep Q-Learning Approach*. [10.1109/ICMLA58977.2023.00155](#)
- **[SymmetrySoftWrist]** Nguyen, et al. (2024). *Symmetry-Aware RL for Robotic Assembly under Partial Observability with a Soft Wrist*. ICRA. [10.1109/ICRA57147.2024.10610103](#)
- **[PolyFit]** Lee, et al. (2024). *PolyFit*. IROS. [10.1109/IROS58592.2024.10802554](#)
- **[CaolInsertion]** Cao, et al. (2024). *On Efficient and Flexible Autonomous Robotic Insertion Assembly in the Presence of Uncertainty*. IEEE RA-L. [10.1109/LRA.2024.3404749](#)
- **[VisionTouch]** Lee, M. A., et al. (2019). *Making Sense of Vision and Touch*. ICRA.

Bimanual and cooperative assembly

- **[BUDS]** Grannen, J., et al. (2023). *Stabilize to Act: Learning to Coordinate for Bimanual Manipulation*. CoRL. [10.48550/arXiv.2309.01087](#)
- **[DualArmSnapFit]** Kumar, et al. (2025). *A Coordinated Dual-Arm Framework for Delicate Snap-Fit Assemblies*. Humanoids. [10.1109/Humanoids65713.2025.11203142](#) / [10.48550/arXiv.2511.18153](#)
- **[A3D]** Liang, et al. (2026). *A3D: Adaptive Affordance Assembly with Dual-Arm Manipulation*. [10.48550/arXiv.2601.11076](#)
- **[ManiGaussian++]** Yu, et al. (2025). *ManiGaussian++*. IROS. [10.1109/IROS60139.2025.11246564](#)
- **[Stavridis]** Stavridis, S., & Doulgeri, Z. (2018). *Bimanual Assembly of Two Parts with Relative Motion Generation*. IROS. [10.1109/IROS.2018.8593928](#) — **the counter-position**
- **[DualArmDexPiH]** Lee, et al. (2022). *Peg-in-Hole Assembly With Dual-Arm Robot and Dexterous Robot Hands*. IEEE RA-L. [10.1109/LRA.2022.3187497](#)
- **[LightVault]** Parascho, S., Han, I. X., Walker, S., Beghini, A., Bruun, E. P. G., & Adriaenssens, S. (2020). *Robotic vault: a cooperative robotic assembly method for brick vault construction*. Construction Robotics 4, 117–126. [10.1007/s41693-020-00041-w](#)
- **[ProgrammedWall]** Bonwetsch, T., Kobel, D., Gramazio, F., & Kohler, M. (2006). *The Informed Wall: Applying Additive Digital Fabrication Techniques on Architecture*. ACADIA 2006, 489–495.

- **[SpatialTimber]** Thoma, A., Adel, A., Helmreich, M., Wehrle, T., Gramazio, F., & Kohler, M. (2018). *Robotic Fabrication of Bespoke Timber Frame Modules*. ROBARCH 2018, 447–458. [10.1007/978-3-319-92294-2_34](#)

Privileged learning and distillation

- **[ProvablePriv]** Cai, et al. (2024). *Provable Partially Observable RL with Privileged Information*. [10.48550/arXiv.2412.00985](#)
- **[StudentInformedTeacher]** Messikommer, N., et al. (2024). *Student-Informed Teacher Training*. [10.48550/arXiv.2412.09149](#)
- **[RealizableStudents]** Kim, et al. (2025). *Distilling Realizable Students from Unrealizable Teachers*. IROS. [10.1109/IROS60139.2025.11247406](#)
- **[InformedAAC]** Ebi, et al. (2025). *Informed Asymmetric Actor-Critic*. [10.48550/arXiv.2509.26000](#)
- **[ACGD]** Srinivasan, et al. (2025). *Visual Multitask Policy Learning with Asymmetric Critic Guided Distillation*. IROS. [10.1109/IROS60139.2025.11247025](#)
- **[VIRAL]** He, et al. (2025). *VIRAL: Visual Sim-to-Real at Scale for Humanoid Loco-Manipulation*. [10.48550/arXiv.2511.15200](#) — **the compute warning**

Snap-fit and interference-fit mechanics

- **[MechanicsSnapFit]** Yoshida, et al. (2020). *Mechanics of a Snap Fit*. Physical Review Letters 125(19), 194301. [10.1103/PhysRevLett.125.194301](#)
- **[LammleSnapFit]** Lämmle, et al. (2022). *Extension of Established Modern Physics Simulation for the Training of Robotic Electrical Cabinet Assembly*. Procedia CIRP. [10.1016/j.procir.2022.05.151](#)
- **[YoonTightTol]** Yoon, et al. (2022). *Fast and Accurate Data-Driven Simulation Framework for Contact-Intensive Tight-Tolerance Robotic Assembly*. [10.48550/arXiv.2202.13098](#)
- **[SnapFitFEA1]** Abdul Manan, et al. (2025). *Analyzing Insertion and Retention Forces in Cantilever Snap-Fits*. IJAME. [10.15282/ijame.22.3.2025.3.0960](#)
- **[SnapFitFEA2]** Stefanoaea, et al. (2024). *A Simulation Method for the One-Time Snap-Fit Assembly Process of PA6 GF60 Components*. ETASR. [10.48084/etasr.6715](#)
- **[CompliantFingers]** Hartisch, et al. (2025). *Toward Simulation-Based Optimization of Compliant Fingers for High-Speed Connector Assembly*. IEEE RA-L. [10.1109/LRA.2026.3655310](#)

Platform

- **[cuRobo]** Sundaralingam, B., et al. (2023). *cuRobo: Parallelized Collision-Free Robot Motion Generation*. ICRA.
- **[Factory]** Narang, Y., et al. (2022). *Factory: Fast Contact for Robotic Assembly*. RSS.
- **[AutoMate]** Tang, B., et al. (2024). *AutoMate: Specialist and Generalist Assembly Policies over Diverse Geometries*. RSS.

7.5 Glossary

Term	Meaning
Clutch	The LEGO stud–cavity interference joint; ~0.1 mm radial interference, 8–15 N pull-off per stud
Placer	The arm performing the press-fit insertion (Franka + thin parallel jaw)

Term	Meaning
Stabilizer	The arm bracing the partial structure (Franka/KUKA + Allegro)
PLAI	Policy-level action integrator — accumulate pose targets rather than command absolute poses ([IndustReal])
Weakest joint	The mated connection with the smallest margin to f_break under the predicted insertion wrench
Force budget	An observation given to the policy specifying maximum allowable force ([FORGE])
Gate	A blocking acceptance checkpoint; work does not proceed past a failed gate
Ledger	Append-only JSONL record of every gate, decision, cut and failure
Cold-start agent	An agent with no conversation history, joining via §5.6

7.6 Change log from prior versions

Change	From	To	Driver
Joint model	build Warp kernel from scratch	integrate BrickSim; extend only if it fails to batch	§1.2.1
Stability	graph min-cut heuristic	StableLego force-balance + weakest-joint output	§1.2.2
Sequencing	greedy topological + lookahead	physics-aware action masking	§1.2.3
Target structures	hand-authored	sampled from StableLego	D14
Bracing	naive nearest-support heuristic (utility)	weakest-joint assignment (contribution)	§1.6
Primary RL run	end-to-end dense reward	residual + sparse reward	§1.4.2
Force handling	reward penalty only	force-budget conditioning	[FORGE]
Verification	hand-coded threshold	learned success predictor	[FORGE]
Vision phase	scheduled	conditional on A3, run first	§1.4.1, §1.5.2
Generative input	stretch goal	deleted	§1.2.4
Calibration	VBD deformable reference (novel)	analytic [MechanicsSnapFit] fit; deformable optional, 3-day cap	§1.5.3
Accessibility claim	"under-explored contribution"	retracted ; cite [BricksToBots]	§1.2.5

Change	From	To	Driver
A6	one ablation among ten	main experiment	§1.6

End of document. Version 3.0. Amendments MUST be recorded in the ledger and reflected in §7.6.